NutriVision – AI-Powered Indian Food Nutrition Estimator

BS IN DATA SCIENCE AND APPLICATIONS, IIT MADRAS
GROUP 2

Problem Statement

Content

Estimating nutritional values from Indian food images is challenging because:

- Indian meals often contain multiple food items in one plate
- Many dishes are visually similar
- Food portions vary significantly
- There is no structured end-to-end dataset for detection + weight + nutrition

Objective

Content

Our system aims to:

- Detect multiple food items from a single image
- Estimate portion size (weight in grams)
- Calculate nutrition automatically
- Build a fully automated end-to-end pipeline

MobileNetV2 - Performance

Model Training Score

- Accuracy : 0.72
- Precision : 0.71
- Recall : 0.71
- F1-score. : 0.69

Key Insights

- Moderate overall performance (~72% accuracy)
- F1-score is slightly lower than accuracy
- Model predictions are fairly consistent with no strong bias toward false positives or false negatives

YOLOv26n - Performance

Training Performance

- Precision: 0.80
- Recall: 0.73
- mAP@50: 0.81
- mAP@50-95: 0.64

Test Results (Final Performance)

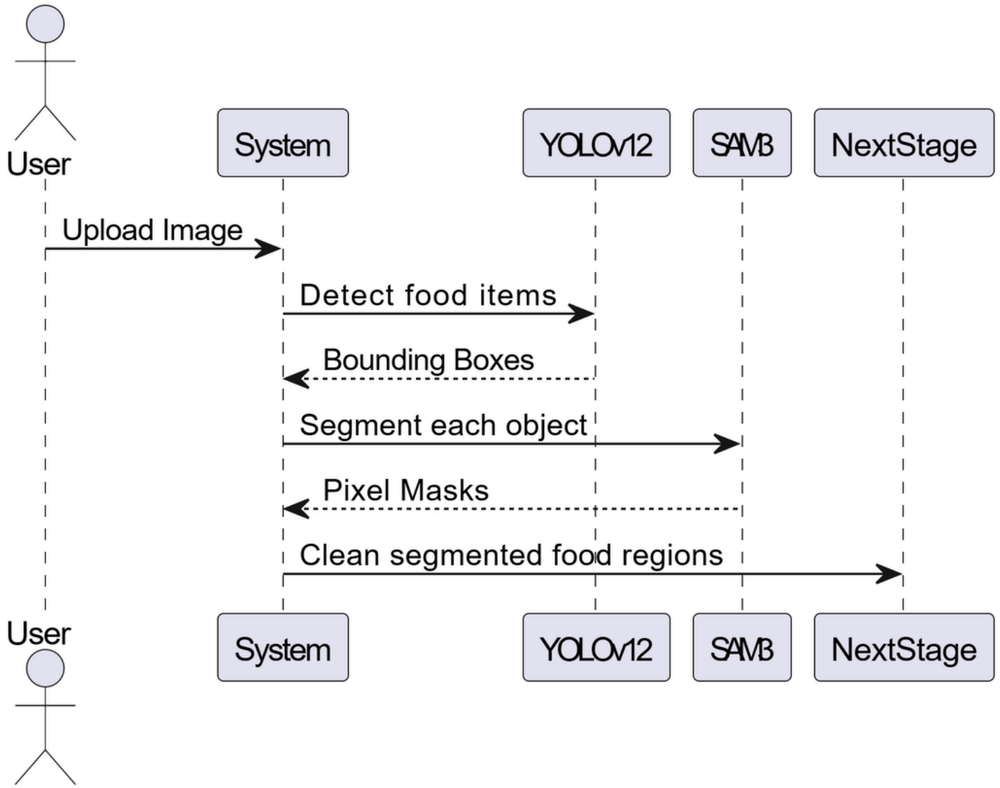
- Precision: 0.81
- Recall: 0.77
- mAP@50: 0.84
- mAP@50-95: 0.67

Key Observations

- Balanced precision & recall
- Test performance is highest, but overall very low. Could be better
- Recall is still not very high

Yolo v12 Performance

& old pipeline



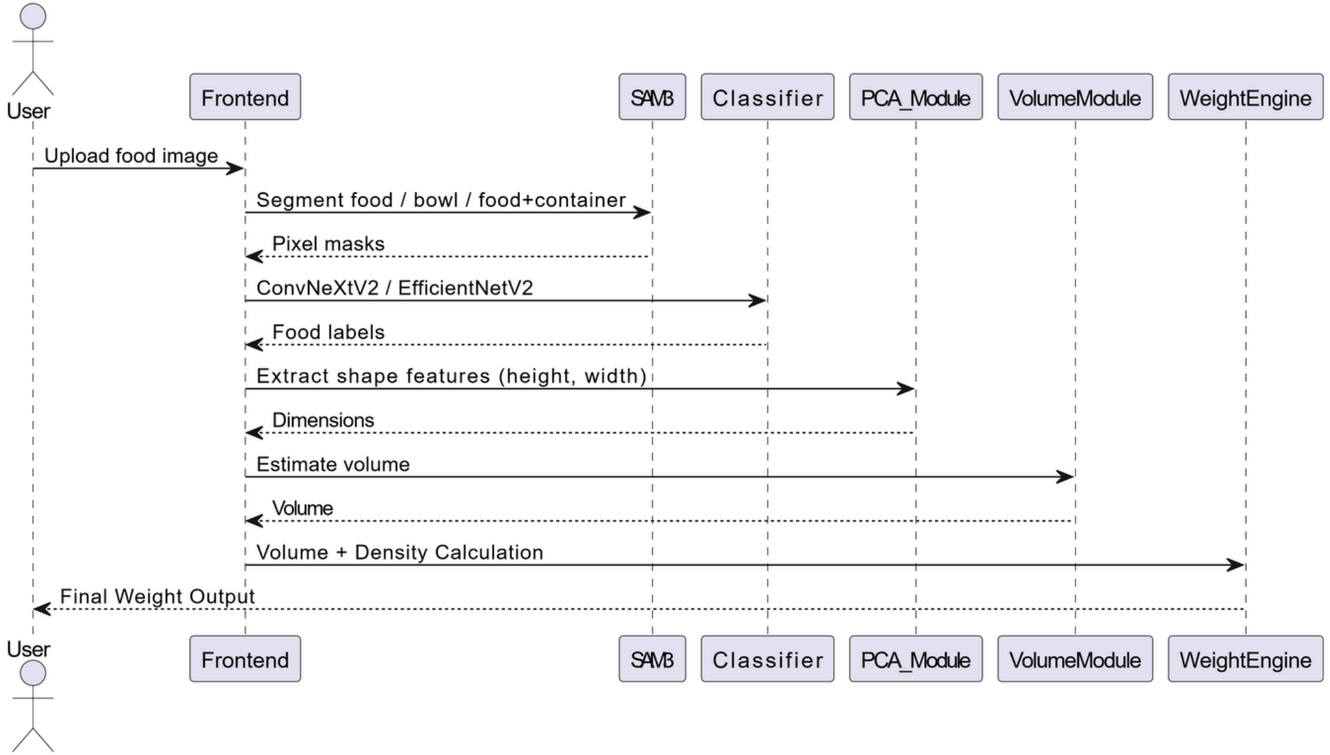
Step 1: Object Detection (YOLOv12)

- Detects multiple food items
- Trained on 111 Indian food classes
- Performance:
- mAP@50 → 89.1%
- Precision → 87.7%

Step 2: Segmentation (SAM 3)

- Converts bounding boxes → pixel-level masks
- Enables accurate area estimation
- Handles irregular shapes (rice, curry, etc.)

Final Pipeline



Why YOLOv12 Removed?

- Detection accuracy was low for food items
- Struggles with:
- Overlapping food
- Similar-looking dishes
- Bounding boxes were not precise enough for weight estimation

New Approach (Improved Pipeline)

- SAM3 Segmentation
- Directly segments:
- Food
- Bowl
- Food + container
- Gives precise pixel-level masks
- Classification
- ConvNeXtV2 / EfficientNetV2
- Accuracy: 92.38%
- PCA (Principal Component Analysis)
- Extracts:
  - Height
  - Width
- Helps estimate object dimensions
- Volume Estimation
- Uses geometric approximation
- Weight Calculation
- Formula
- Weight=Volume×Density

Classification Models

Why Classification after Detection?

- Detection may confuse similar foods (e.g., biryani variants)
- Classification improves fine-grained accuracy

Model 1: ConvNeXtV2

- Trained for 2 epochs (disk limitation)
- Accuracy: ~90%
- Heavy model → high storage usage

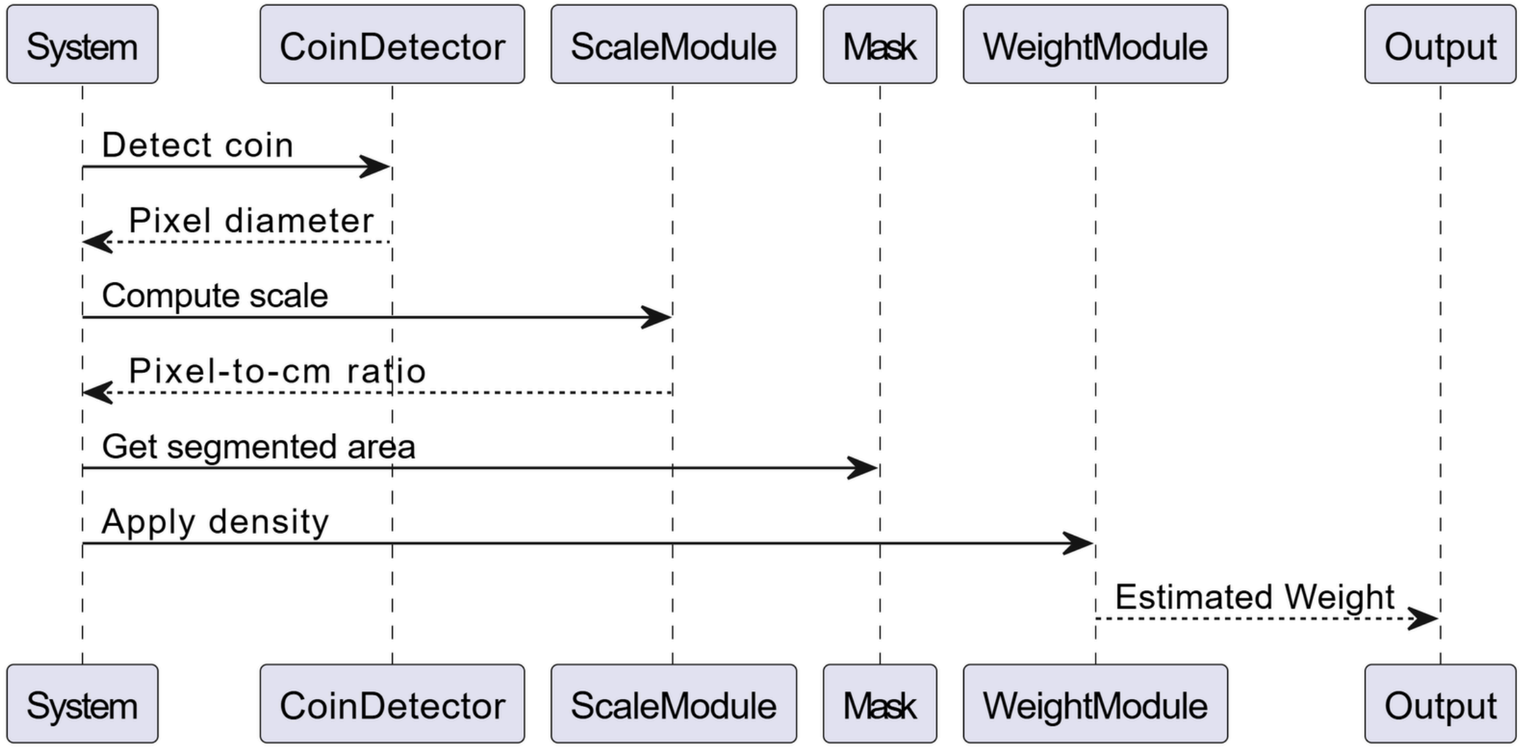
Model 2: EfficientNetV2

- Trained for 7 epochs (time constraint)
- Accuracy: 92.38% (Best)
- Training Result
- Epoch 8:
- Validation Accuracy: 92.58%
- Loss stabilized → good convergence

Key Insight

- EfficientNetV2 outperformed ConvNeXtV2 due to:
- Better efficiency
- More training epochs

Scale + Weight Estimation



Reference Object

- ₹10 coin (diameter = 2.7 cm)

Steps

- Detect coin in image
- Compute pixel-to-cm ratio
- Calculate food area from segmentation mask
- Convert area → volume → weight

Weight Estimation Logic

- Area (pixels) → Area (cm^2)
- Use density assumption:
- Rice ≠ Curry ≠ Bread
- Final:
- Weight ≈ Area × Thickness × Density

Hyperparameter Tuning

will be completed before 29 March on classification model

THANKYOU